# SentryScan: Assignment 1.1 — PEAS & Environment Modelling

**Course:** 21CSE525T — Agent Technology
**Assignment:** 1.1 — PEAS & Environment Modelling
**Student:** R Ganesh [RA2212704010031]
**Repository:** https://github.com/drgeekg/SentryScan
**Target Branch:** `assignment/1.1-peas-environment`
**Git Tag:** `assignment-1.1`
**Status:** Completed & Verified

---

## 1. Executive Summary & Theoretical Context

Assignment 1.1 establishes the foundational theoretical, behavioral, and architectural specifications for **SentryScan**—an authorized autonomous vulnerability scanner agent simulation designed for synthetic telemetry analysis.

### 1.1 Problem Statement & Agent Persona

Modern enterprise security teams face severe alert fatigue and the operational hazards of intrusive live vulnerability scanning. SentryScan acts as an **Autonomous Defensive Telemetry Analyst**: - **Defensive Posture:** Operates strictly on *perceived telemetry* rather than active exploitation. - **Safety Boundary:** Synthetic simulation only. All host addresses strictly use IETF documentation ranges (RFC 5737: `192.0.2.0/24`, `198.51.100.0/24`, `203.0.113.0/24`; RFC 3849: `2001:db8::/32`). SentryScan never attempts live socket connections to these targets. - **Untrusted Telemetry Guarantee:** All banners and external strings are treated as untrusted data inputs, precluding injection attacks or arbitrary execution.

### 1.2 Agent vs. Conventional OOP Paradigm (Russell & Norvig, Ch. 2; Wooldridge)

- **Control Autonomy:** SentryScan owns its thread of control. It evaluates resource budgets and operational utility before deciding whether to accept tasks, unlike passive OOP objects which execute methods unconditionally upon invocation.
- **Cognitive Mental State:** SentryScan maintains epistemic mental states (Beliefs, Desires, Intentions) rather than raw encapsulated class properties.
- **Speech-Act Communication:** Inter-agent interactions use formal performatives (`CFP`, `PROPOSE`, `ALERT`, `CONFIRM`) rather than rigid remote procedure calls (RPC).

---

## 2. Architecture & Implementation Specification

### 2.1 Formal PEAS Framework

| Component | Specification for SentryScan |
|---|---|
| **Performance Measure ($P$)** | - **Detection Accuracy:** High precision and recall on genuine exposures without false alarms on benign services.<br>- **Data Integrity Defense:** Detection rate of poisoned/mutated records without access to ground truth.<br>- **System Stability & Latency:** Minimal queue turnaround time; bounded memory footprint; zero dropped events.<br>- **Protocol Compliance:** Handshake resolution rate in multi-agent negotiation; bounded rounds ($\le 5$); zero unauthorized message acceptances.<br>- **Audit Completeness:** 100% gap-free event sequence persistence ( `seq` ). |
| **Environment ($E$)** | - **Simulated Endpoint Fleet:** Heterogeneous synthetic hosts ( `sim-host-001` … `sim-host-N` ) generating concurrent telemetry.<br>- **Telemetry Stream:** Ingestion queue carrying normal baseline records, malformed payloads, and deliberately mutated (poisoned) records.<br>- **Communication Channels:** Loopback TCP sockets ( `127.0.0.1` ) and in-process message queues.<br>- **Knowledge Repositories:** Read-only CVE / NVD data interfaces and local offline caches. |
| **Actuators ($A$)** | - **Finding Publisher:** Emits structured `Finding` objects with deterministic severity, evidence links, and confidence scores.<br>- **Event Recorder:** Commits append-only chronological `AgentEvent` entries with monotonic `seq` to JSONL and SQLite sinks.<br>- **Envelope Dispatcher:** Constructs, signs (Ed25519), and transmits inter-agent messages ( `ALERT` , `PARAMS_UPDATE` , `CFP` , `PROPOSE` , `ACCEPT` ).<br>- **BDI Intention Actuator:** Modifies active goals, triggering plan revision or resource reallocation.<br>- **Report Actuator:** Generates structured JSON/CSV audit reports and Rich console dashboards. |
| **Sensors ($S$)** | - **Telemetry Sensor:** Stream receiver that parses and validates raw telemetry into typed `TelemetryRecord` objects.<br>- **Integrity Sensor:** Rule-based parser checking timestamp drift, hash duplication, service-banner mismatch, and prompt injection strings.<br>- **Socket / Network Sensor:** TCP frame listener decoding length-prefixed JSON envelopes and validating security credentials.<br>- **Knowledge Sensor:** Client querying local CVE cache and external NVD endpoints for vulnerability enrichment. |

### 2.2 Environment Classification (Russell & Norvig 7 Dimensions)

1. **Partially Observable:** The agent observes only surface-level telemetry emitted by devices (ports, service names, version banners). It cannot inspect remote kernel memory or filesystems, and ground-truth poisoning labels are hidden in an isolated evaluator sidecar.
2. **Stochastic:** While internal agent reasoning is deterministic, the external telemetry stream introduces variable queue latency, concurrent thread interleaving, synthetic network jitter, and probabilistic injection operators.
3. **Sequential:** Decisions constrain future states: suspect record flags update device risk baselines, findings alter BDI beliefs, and task awards consume execution budgets.
4. **Dynamic:** The environment evolves continuously as background daemons generate telemetry while scanner workers deliberate.
5. **Discrete:** Network ports ($1 \dots 65535$), performatives ( `ALERT` , `CFP` , etc.), severities ( `LOW` to `CRITICAL` ), and event sequences ($seq \in \mathbb{N}$) are discrete.
6. **Multi-Agent:** Supervisor, Scout, Triage, and Analyst agents interact with distinct utilities via Contract-Net negotiation.
7. **Known:** Environmental laws (schemas, rules, protocols, utility equations) are fully known to the agent design.

---

## 3. Rubric Compliance & Verification Matrix

| Evaluation Criterion | Specific Requirement | Specification Section | Evidence Artifact & Verification |
|---|---|---|---|
| **1. Agent Persona & Problem Formulation** | Clear definition of agent persona, problem context, and operational safety boundary | Section 1: Persona & Problem Statement | `deliverables/SentryScan_Design_Specification.md#L1-L36` `deliverables/SentryScan_Design_Specification.pdf` (Pages 1–2) |
| **2. Formal PEAS Framework** | Comprehensive breakdown of Performance measure ($P$), Environment ($E$), Actuators ($A$), and Sensors ($S$) with quantifiable metrics | Section 2: PEAS Framework Formulation | `deliverables/SentryScan_Design_Specification.md#L38-L58` `deliverables/SentryScan_Design_Specification.pdf` (Pages 2–3) |
| **3. Environment Classification** | Rigorous evaluation across all 7 Russell & Norvig dimensions with theoretical and domain grounding | Section 3: Environment Classification | `deliverables/SentryScan_Design_Specification.md#L60-L80` `deliverables/SentryScan_Design_Specification.pdf` (Pages 3–4) |
| **4. Agent Architecture vs. OOP Comparison** | Detailed architectural comparison citing autonomy, cognitive states, situatedness, and speech acts, referencing the Unit-1 paradigm | Section 4: Agent Architecture vs. Conventional OOP | `deliverables/SentryScan_Design_Specification.md#L82-L135` `deliverables/SentryScan_Design_Specification.pdf` (Pages 5–6) |
| **5. Architectural & Data Flow Diagrams** | Clear visual diagrams depicting perception, integrity checks, rule engine, and persistent storage | Section 5: Initial Architecture & Data Flow | `deliverables/diagrams/architecture.svg` `deliverables/diagrams/environment_model.svg` `deliverables/SentryScan_Design_Specification.pdf` (Pages 7–8) |
| **6. Future Multi-Agent Extensions** | Concrete expansion roadmap linking reflex agent to concurrency, sockets, LangGraph, utility, and BDI | Section 6: Future Multi-Agent & Deliberative Extensions | `deliverables/SentryScan_Design_Specification.md#L137-L180` `deliverables/SentryScan_Design_Specification.pdf` (Pages 8–9) |
| **7. Submission & Packaging Integrity** | Clean git history, merge commit, immutable tag, and verified archive packaging | Monorepo root and git history | Git Tag: `assignment-1.1` Archive: `dist/SentryScan_assignment_1_1.zip` |

## 4. Execution Results & Empirical Evidence

- **Specification Deliverable:** Complete 187-line comprehensive design document compiled to pixel-perfect vector PDF.
- **Architectural Diagrams Generated:**
- `architecture.svg`: Sense-Parse-Decide-Act data pipeline and persistent event bus.
- `environment_model.svg`: Telemetry ingestion boundary, trust perimeter, and documentation address space.
- **Verification Checklist:** All 7 acceptance items verified in `assignments/assignment_1_1/evidence/checklist.md`.

## 5. Submission Package & Reproduction Instructions

### 5.1 Exact Submission Components

1. **Report Document:** `assignments/assignment_1_1/deliverables/SentryScan_Assignment_1_1_Report.pdf`
2. **Repository URL:** `https://github.com/drgeekg/SentryScan`
3. **Target Branch:** `assignment/1.1-peas-environment`
4. **Immutable Git Tag:** `assignment-1.1`
5. **Packaged ZIP Archive:** `dist/SentryScan_assignment_1_1.zip`
6. **SHA-256 Checksum:** `dist/SentryScan_assignment_1_1.zip.sha256`

### 5.2 Verification Commands

```
# Re-generate specification HTML and PDF
python scripts/export_specification_pdf.py

# Package submission archive from immutable tag
python scripts/package_assignment.py assignment-1.1
```